

Relatório de Evolução do Projeto

ChargeGrid Assistant — GoodWe EV Challenge 2026 — Sprint 03

Integrantes: Pedro Ferreras (RM 568713) · Pedro Santos (RM 571017) · Victor Binot (RM 571499) · Gustavo Kunitaki (RM 571400) · Kauanne Oliveira (RM 574191) · Nayhely Estela (RM 571416)

1. Resumo da evolução

Nas Sprints 1 e 2, o ChargeGrid Assistant foi entregue como um chatbot funcional em Python: loop de terminal simples (`chatbot.py`), conectado diretamente ao client da lib `ollama`, com histórico de conversa mantido como uma lista reenviada a cada chamada, few-shot embutido no meio do system prompt, e log das conversas em arquivo `.txt`.

Na Sprint 03, o núcleo conversacional foi reconstruído em LangChain (LCEL), mantendo o `chatbot.py` original intacto no repositório como registro histórico. O que existe agora, além do legado:

- Chain declarativa prompt | llm | parser (`src/chain/builder.py`)
- Memória por sessão com corte por tokens (`src/chain/memoria.py`)
- Saída estruturada validada com Pydantic v2 (ConsultaRecarga, com `field_validator`)
- System prompt versionado (v1 legado × v2 com context engineering / tags XML)
- Guardrails determinísticos de escopo e jailbreak (`src/guardrails/`)
- Eval set formal com 12 casos (happy path, edge case, jailbreak, out-of-scope) e execução automatizada
- Comparação entre modelos (gpt-oss:120b × qwen3:8b) com parâmetros documentados
- Entry point próprio para a versão nova (`chatbot_lcel.py`), mantendo o `chatbot.py` original funcional

2. Refatoração — decisões técnicas e trade-offs

Memória de sessão

O enunciado pedia `RunnableWithMessageHistory + ConversationTokenBufferMemory`. Ao testar, verificamos que `ConversationTokenBufferMemory` foi removida das versões atuais do LangChain. Trade-off: usar uma versão antiga da biblioteca (arriscando outras incompatibilidades) ou adotar a alternativa atual recomendada pelo próprio LangChain, `trim_messages`. Optamos pela segunda, priorizando compatibilidade com a versão estável da lib sobre bater literalmente com o nome da classe citada no enunciado.

Saída estruturada separada da conversa livre

Em vez de forçar toda resposta a passar por um schema Pydantic (o que quebraria perguntas abertas), criamos duas chains: uma conversacional (`construir_chain_conversacional`) e outra estruturada (`consultar_estruturado`, via `with_structured_output`). Trade-off: mais uma função para manter, mas evita forçar formato onde não se aplica.

Guardrails em código, além do prompt

O bloco `<guardrails>` do prompt v2 já instrui o modelo a recusar jailbreak e temas fora de escopo, mas isso não é garantido (modelo é probabilístico). Adicionamos uma camada determinística em Python (`scope_validator.py`, `moderation.py`) que roda antes da chamada ao LLM. Trade-off: detecção por regex/keyword não cobre 100% dos casos (um atacante criativo pode reformular), mas cobre os padrões mais comuns sem custo de API.

3. Comparativo antes/depois

Métrica	Sprints 1/2 (legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota no eval)	Testes manuais informais (docs/testes.md), qualitativo "Adequada" nos casos testados	12 casos formais no eval set; 6/6 (100%) nos casos determinísticos de guardrail
Tokens por turno (prompt, proxy aprox.)	~217 palavras (v1)	~322 palavras (v2) — maior por incluir bloco de guardrails
Latência média por resposta	Não medida formalmente	~1.08s (média de 3 chamadas reais, gpt-oss:120b)
Acurácia do structured output	Não se aplica (sem saída estruturada)	100% nos 3 cenários de teste do schema (1 válido + 2 inválidos capturados corretamente)

Nota: os valores de tokens são uma aproximação por contagem de palavras (ver prompts/versoes.md) — o tokenizer real do gpt-oss:120b não é acessível via tiktoken. A latência do legado não foi medida porque o chatbot.py da Sprint 2 não registrava esse dado; ambas as versões usam a mesma API do Ollama Cloud, então a diferença de latência entre elas, se houver, viria do overhead do LangChain, não do modelo em si.

4. Problemas encontrados e soluções

1. ConversationTokenBufferMemory removida do LangChain

O enunciado citava essa classe, mas ela não existe nas versões atuais da lib (existia em versões antigas, tipo 0.2.x). Decisão: usar trim_messages, a alternativa atual recomendada pelo próprio LangChain para o mesmo objetivo (memória por sessão com limite de tokens).

2. RunnableWithMessageHistory depreciado

Ao rodar a chain, o LangChain avisa que essa classe está sendo substituída por LangGraph. Decisão: manter RunnableWithMessageHistory mesmo assim, já que o enunciado explicitamente coloca LangGraph fora do escopo desta sprint.

3. Modelo qwen3:8b indisponível na conta Ollama Cloud (erro 404)

Ao rodar a comparação de modelos, o segundo modelo sugerido pelo enunciado não estava disponível na conta usada. Decisão: documentar o erro real no relatório de modelos e manter o script preparado para rodar assim que o modelo estiver disponível (ou for substituído por outro confirmado).

4. Inconsistência entre documentação e código sobre few-shot

Um comentário inicial no prompt v2 afirmava que os exemplos few-shot seriam reintroduzidos na chain via FewShotChatMessagePromptTemplate. Isso nunca foi implementado. Decisão: corrigir a documentação (prompts/versoes.md) em vez de manter uma afirmação que não correspondia ao código real, já que few-shot na chain não é exigido pelo enunciado.

5. Equipe e divisão de trabalho

Integrantes: Pedro Ferreras (RM 568713), Pedro Santos (RM 571017), Victor Binot (RM 571499), Gustavo Kunitaki (RM 571400), Kauanne Oliveira (RM 574191), Nayhely Estela (RM 571416).

Nesta Sprint 03, a implementação técnica (refactory LangChain, guardrails, schema Pydantic, evals e relatórios) foi conduzida por Victor Binot. Antes da entrega, o restante do grupo (Pedro Ferreras, Pedro Santos, Gustavo Kunitaki, Kauanne Oliveira e Nayhely Estela) revisou e validou o material entregue.